Things are a bit different in the `AuthenticationMiddleware`. Instead of deserializing a cookie containing the `ClaimsPrincipal`, the middleware decodes the JWT token in the `Authorization` header. It validates the signature using the signing keys from the identity provider, and verifies that the audience has the expected value and that the token has not expired.

If the token is valid, the authentication middleware creates a `ClaimsPrincipal` representing the authenticated request and sets it on `HttpContext.User`. All middleware after the authentication middleware sees the request as authenticated.

TIP If the claims in the token don't match the key values you're expecting, you can use *claims transformation* to remap claims. This applies to cookie authentication too, but it's particularly common when you're receiving tokens from third-party identity providers, where you don't control the names of claims. You can also use this approach to add extra claims for a user, which weren't in the original token. To learn more about claims transformation, see http://mng.bz/gBJV.

We've covered a lot of theory about JWT tokens in this chapter, so you'll be pleased to hear it's time to look at some code!

25.3 Adding JWT bearer authentication to minimal APIs

In this section you'll learn how to add JWT bearer token authentication to an ASP.NET Core app. I use the minimal API Recipe API application we